Reviewer Pack — Convex Body Volume Lower Bounds Disjointness Communication C…

Entry 70f3d3ccad21 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 13:46:12 UTC

Convex Body Volume Lower Bounds Disjointness Communication Complexity

Entry ID: `70f3d3ccad21` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 13:46:12 UTC

1. Conjecture

Field A (mathematical branch): Geometry of Numbers **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For every n ≥ 1, the communication complexity of DISJ_n is at least log(V(n)), where V(n) is the volume of the convex body defined by {x ∈ [0,1]^n | ∃y ∈ [0,1]^n, x_i + y_i ≤ 1 ∀i} under the □₁-norm. This bound is tight up to polylog(n) factors.

Rationale (proposer's reasoning):

> The geometry of numbers provides natural measures of 'complexity' for high-dimensional convex bodies. By linking the communication matrix's structure to a convex body's volume, we expose a geometric invariant that could reveal inherent information-theoretic barriers in solving DISJ_n.

Taxonomy category: `COMM_DISJ` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0c87f4beb29812ac`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=7.8s

5.1 Generated Python source

```python
import random
import math
from itertools import product

def generate_disj_instance(n):
    return [random.choice([0, 1]) for _ in range(n)]

def monte_carlo_volume(n, samples=100000):
    volume = 0
    for _ in range(samples):
        point = [random.random() for _ in range(n)]
        if all(point[i] + point[j] <= 1 for i, j in product(range(n), repeat=2)):
            volume += 1
    return volume / samples

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    total_volume = 0

    for _ in range(instances_tested):
        instance = generate_disj_instance(n)
        volume = monte_carlo_volume(n)
        if volume == 0:
            return {
                "metric_name": "log(V(n))",
                "metric_value": None,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": "volume_is_zero"
            }
        total_volume += volume

    average_volume = total_volume / instances_tested
    communication_complexity = n

    if communication_complexity >= math.log(average_volume):
        conjecture_holds = True
```

```python
            counterexample = ""
        else:
            conjecture_holds = False
            counterexample = f"communication_complexity={communication_complexity}, log(V(n))={math.lo

        return {
            "metric_name": "log(V(n))",
            "metric_value": math.log(average_volume),
            "instances_tested": instances_tested,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds': False,
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'
TRIAL: {'metric_name': 'log(V(n))', 'metric_value': None, 'instances_tested': 30, 'conjecture_holds'

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_080a7c92.py", line 83, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_080a7c92.py", line 83, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to KeyError before producing valid results; cannot confirm conjecture status. | next: Fix the test's seed handling and re-run with proper error handling for metric computation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	103279
2	preregistration	ollama_remote	qwen3:8b	0	0	24146
3	novelty	ollama_remote	qwen3:8b	0	0	21312
4	novelty	ollama_remote	qwen3:8b	0	0	15545
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9967
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7536
7	judge	ollama_remote	qwen3:8b	0	0	16149

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 197935 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/70f3d3ccad21.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/70f3d3ccad21.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/70f3d3ccad21.tar.gz` (if generated)